EVALUATION

Evaluation custom Node2Vec vs Node2Vec gốc

Đối chiếu embedding qua main.py và test.py

Karate Club · Logistic Regression · Accuracy và F1

1. Mục tiêu đánh giá

Node2Vec là phương pháp học biểu diễn không giám sát: trong lúc học embedding, model không dùng nhãn "Mr. Hi" hay "Officer". Evaluation dùng nhãn chỉ sau khi embedding đã được tạo để kiểm tra embedding có chứa thông tin hữu ích cho một downstream task hay không.

Evaluation không so sánh trực tiếp từng tọa độ vector. Hai mô hình có thể tạo embedding tương đương nhưng bị xoay, phản chiếu hoặc đổi hệ trục. Thay vào đó, so sánh khả năng phục vụ cùng một nhiệm vụ phân loại.

2. Chuẩn bị trong main.py

2.1. Reproducibility và graph

```
seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)

G = nx.karate_club_graph()
```

Code dùng seed 42 cho Python, NumPy và PyTorch. Graph là Zachary Karate Club của NetworkX, gồm 34 node và 78 edge. Nhãn câu lạc bộ đã nằm trong thuộc tính node nhưng chưa được dùng khi train embedding.

2.2. Hyperparameter chung

Tham số | Giá trị | Vai trò

Embedding dimension | 128 | Số chiều mỗi node vector

Walks per node | 10 | Số random walk bắt đầu từ mỗi node

Walk length | 80 | Số bước walk theo cấu hình project

Context/window | 5 | Phạm vi context khi học Skip-gram

p, q | 1.0, 1.0 | Random walk không bias return/in-out

Negative samples | 5 | Chỉ áp dụng rõ ràng trong custom trainer

Batch size | 256 | Kích thước batch custom

Epochs | 1 | Số vòng train trong lần chạy hiện tại

Learning rate | 0.001 | Learning rate cho Adam custom

3. Sinh embedding custom

```
node2vec = Node2Vec(
    G=G, p=p, q=q,
    dimension=embedding_dim,
    walks_per_node=walks_per_node,
    walk_length=walk_length,
    context_size=context_size,
)

custom_embeddings = node2vec.fit(
    num_negative=num_negative,
    batch_size=256,
    epochs=1,
    lr=0.001,
    device=device,
)
```

fit() của custom implementation thực hiện toàn bộ pipeline:

• Tiền tính transition probabilities từ p, q.

• Tạo 10 × |V| random walks.

• Tạo positive pairs từ các node trong walk.

• Tạo 5 negative contexts cho mỗi positive pair theo tần suất node0.75.

• Map node label sang integer ID và tạo TensorDataset.

• Train hai bảng embedding bằng Skip-gram, negative-sampling loss và Adam.

• Trả về source_embedding.weight.

Trong kết quả trả về, dòng thứ i tương ứng với node thứ i trong list(G.nodes()).

4. Sinh embedding từ package gốc

```
original_node2vec = OriginalNode2Vec(
    graph=G,
    dimensions=embedding_dim,
    walk_length=walk_length,
    num_walks=walks_per_node,
    p=p,
    q=q,
    workers=1,
    quiet=True,
    seed=seed,
)

original_model = original_node2vec.fit(
    window=context_size,
    min_count=1,
    batch_words=256,
    epochs=1,
    seed=seed,
)

original_embeddings = np.asarray([
    original_model.wv.get_vector(str(node))
    for node in G.nodes()
])
```

Package gốc tạo model Skip-gram thông qua stack của package node2vec và gensim. Code lấy vector bằng original_model.wv.get_vector(str(node)), rồi xếp theo đúng thứ tự G.nodes() để hai ma trận có cùng ý nghĩa theo hàng.

Custom | Package gốc

Tự tạo walk, positive/negative samples và PyTorch model | Package tự quản lý walk và gensim training

Negative sampling cố định num_negative=5 | Negative sampling do gensim configuration mặc định/implementation đảm nhiệm

Output là source embedding table | Output lấy từ model.wv

Mapping dùng node label trực tiếp trong node2id | Vector được truy vấn bằng chuỗi str(node)

5. Lưu kết quả và bảo toàn mapping

```
np.save("karate_embeddings_custom.npy", custom_embeddings)
np.save("karate_embeddings_original.npy", original_embeddings)
np.save("karate_node_ids.npy", np.asarray(list(G.nodes()), dtype=object))
```

Ba file này có vai trò:

File | Shape/kiểu | Ý nghĩa

karate_embeddings_custom.npy | (34, 128) | Embedding từ implementation custom

karate_embeddings_original.npy | (34, 128) | Embedding từ package gốc

karate_node_ids.npy | (34,) | Node label ứng với từng row embedding

6. Chuẩn bị nhãn trong test.py

```
G = nx.karate_club_graph()
node_list = list(G.nodes())
node_ids = np.load("karate_node_ids.npy", allow_pickle=True).tolist()

if node_ids != node_list:
    raise ValueError("Embedding node IDs do not match the graph node order.")

y = np.array([
    0 if G.nodes[node]["club"] == "Mr. Hi" else 1
    for node in node_list
])
```

Evaluation chuyển nhãn câu lạc bộ thành bài toán nhị phân:

Club | Label

Mr. Hi | 0

Club còn lại | 1

Đây là nhãn ground truth chỉ dùng trong bước downstream evaluation, không rò rỉ vào quá trình sinh embedding.

7. Chia train/test công bằng

```
splitter = StratifiedShuffleSplit(
    n_splits=30,
    test_size=0.3,
    random_state=42,
)
```

StratifiedShuffleSplit tạo 30 lần chia ngẫu nhiên; mỗi lần dùng 70% node để train và 30% node để test, đồng thời giữ tỷ lệ hai class gần giống nhau ở hai tập.

Đối với mỗi embedding file, code gọi cùng splitter và cùng nhãn y. Vì splitter có random_state=42, hai phương pháp được đánh giá trên cùng các index train/test tương ứng.

Mỗi lần lặp: embedding X → X_train/X_test → classifier.fit → classifier.predict → metrics

8. Classifier và metrics

8.1. Logistic Regression

```
clf = LogisticRegression(
    max_iter=1000,
    random_state=42
)

clf.fit(X_train, y_train)
predictions = clf.predict(X_test)
```

Logistic Regression là linear probe: nó kiểm tra liệu thông tin phân biệt hai club có thể được tách bằng một biên tuyến tính trên embedding hay không. Classifier không được phép thấy test labels trong lúc fit.

8.2. Accuracy

Accuracy = số dự đoán đúng / tổng số mẫu test

Accuracy dễ đọc và phù hợp khi hai class không quá lệch. Giá trị càng gần 1 càng tốt.

8.3. F1 score

F1 = 2 × Precision × Recall / (Precision + Recall)

Code gọi f1_score(y_test, predictions) với thiết lập mặc định của sklearn cho bài toán nhị phân. F1 cân bằng precision và recall, hữu ích khi muốn tránh model chỉ dự đoán tốt một class.

8.4. Mean và standard deviation

```
accuracies.append(accuracy_score(y_test, predictions))
f1_scores.append(f1_score(y_test, predictions))

mean = np.mean(scores)
std = np.std(scores)
```

Kết quả cuối là mean ± std trên 30 split:

• Mean cao: hiệu năng trung bình tốt.

• Std thấp: kết quả ổn định hơn khi thay đổi tập train/test.

• Mean cao nhưng std lớn: có thể phụ thuộc mạnh vào node được chọn vào train set.

9. Kết quả hiện tại

Theo cấu hình hiện tại được ghi trong README của project:

Embedding | Accuracy | F1

Custom | 0.9788 ± 0.0385 | 0.9807 ± 0.0352

Original | 0.9788 ± 0.0385 | 0.9797 ± 0.0369

Diễn giải thận trọng: trong lần chạy này, hai phương pháp có Accuracy trung bình bằng nhau; F1 của custom nhỉnh hơn rất nhẹ. Chênh lệch nhỏ và độ lệch chuẩn còn đáng kể so với chênh lệch mean, nên không nên kết luận custom luôn tốt hơn chỉ từ một graph nhỏ.

10. Lệnh chạy end-to-end

```
# Sinh hai bộ embedding và lưu .npy
python main.py

# Đọc hai bộ embedding, train/evaluate classifier 30 lần
python test.py
```

Output của main.py còn in device, shape embedding và loss history custom. Output của test.py in từng phương pháp theo format:

```
Custom    | Accuracy: mean +/- std | F1: mean +/- std
Original  | Accuracy: mean +/- std | F1: mean +/- std
```

11. Checklist để so sánh đúng

Kiểm tra | Trạng thái trong project

Cùng graph | Cùng nx.karate_club_graph()

Cùng dimension | 128

Cùng số walk/node | 10

Cùng p/q và walk length | p=q=1.0, length 80

Cùng context/window | 5

Cùng node order khi xếp rows | Được kiểm tra qua karate_node_ids.npy

Cùng train/test splits | StratifiedShuffleSplit(random_state=42)

Cùng classifier | LogisticRegression(max_iter=1000, random_state=42)

Cùng metrics | Accuracy và F1 trên test set

12. Kết luận

Evaluation của project có hai tầng:

• Training comparison: custom và package gốc cùng tạo embedding từ Karate Club graph với cấu hình tương đương.

• Downstream comparison: mỗi embedding được đưa vào cùng Logistic Regression và đánh giá trên cùng 30 stratified splits.

Do đó, Accuracy/F1 phản ánh mức độ hữu ích của embedding cho phân loại club trong thí nghiệm này, không phải độ giống nhau của hai vector hoặc bằng chứng rằng một implementation đúng tuyệt đối trong mọi graph.

Các file sinh ra bởi main.py: karate_embeddings_custom.npy, karate_embeddings_original.npy và karate_node_ids.npy.